

... REPORTE DE ACTIVIDAD 12

Nombre del proyecto: VALIDATION OF FILE EXTENSION
WITH NULL BYTE BYPASS

Nivel: Apprentice

Nombre: Karla Vanessa Glz Espinoza

TIPO DE EXPLOTACIÓN

Path Traversal (Directory Traversal) con evasión de validación de extensión mediante byte nulo (Null Byte Injection) / CWE-22: Improper Limitation of a Pathname to a Restricted Directory y CWE-158: Improper Neutralization of Null Byte or NUL Character.

OBJETIVO DEL LABORATORIO

Comprender las fallas de seguridad originadas por la discrepancia en el manejo de terminadores de cadena entre lenguajes de alto nivel y llamadas del sistema en C/C++, demostrando cómo la inyección de un byte nulo (%00) permite eludir las restricciones de extensión de archivo requerida (como .png o .jpg) para acceder y divulgar archivos críticos fuera del directorio autorizado (como /etc/passwd).

EXPLICACIÓN TÉCNICA

Muchas aplicaciones web intentan mitigar ataques de Path Traversal o controlar la carga/descarga de recursos validando que el nombre del archivo termine con una extensión permitida (por ejemplo, validando que finalice con .png, .jpg o .jpeg).

Históricamente, y en entornos heredados o configuraciones donde el intérprete interactúa directamente con APIs del sistema de archivos escritas en C (como llamadas POSIX tipo `open()` o `fopen()`), las cadenas de texto son delimitadas por un carácter de terminación nulo (`\0` o ASCII `0x00`). Cuando una aplicación web recibe una entrada codificada en URL que contiene `%00`, el lenguaje de alto nivel puede procesar la cadena completa para validar si concluye con la extensión requerida (ej. `.../etc/passwd%00.png` termina efectivamente en `.png`). Sin embargo, al enviar dicha ruta a la función de apertura del sistema de archivos subyacente, la lectura de la cadena se trunca inmediatamente en el byte nulo, descartando por completo el sufijo posterior (MITRE, 2023).

De acuerdo con CISA (2022) y OWASP Foundation (2023), depender únicamente de la validación sintáctica de sufijos sin saneamiento de caracteres de control ni canonicalización permite que un atacante combine secuencias de escape de directorios (`../`) con la inyección de byte nulo, logrando leer archivos arbitrarios con cualquier extensión sin disparar la alerta del filtro.

PROCEDIMIENTO

Paso 1: Se inició el entorno del laboratorio provisto por Web Security Academy (File path traversal, validation of file extension with null byte bypass) y se configuró Burp Suite Community Edition para interceptar el tráfico HTTP del navegador integrado.

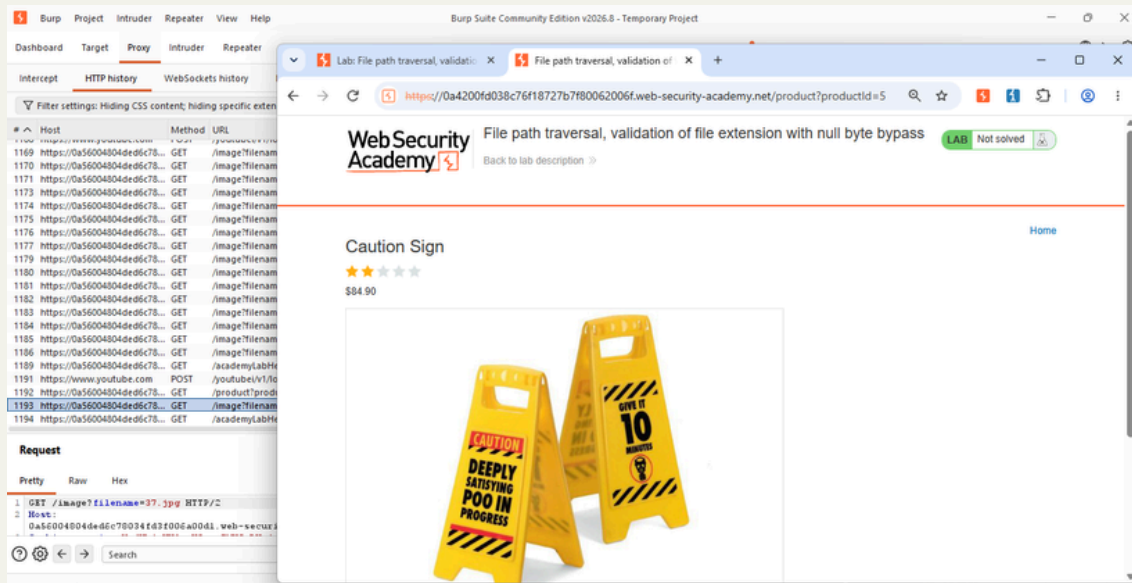


Figura 12.1 Pagina principal de la tienda en un apartado de “View details” de un producto.

Paso 2: En la pestaña Proxy > HTTP history, se localizó la solicitud HTTP dirigida al endpoint /image encargada de recuperar las imágenes de los productos (parámetro filename), observando que utiliza extensiones de imagen estándar.

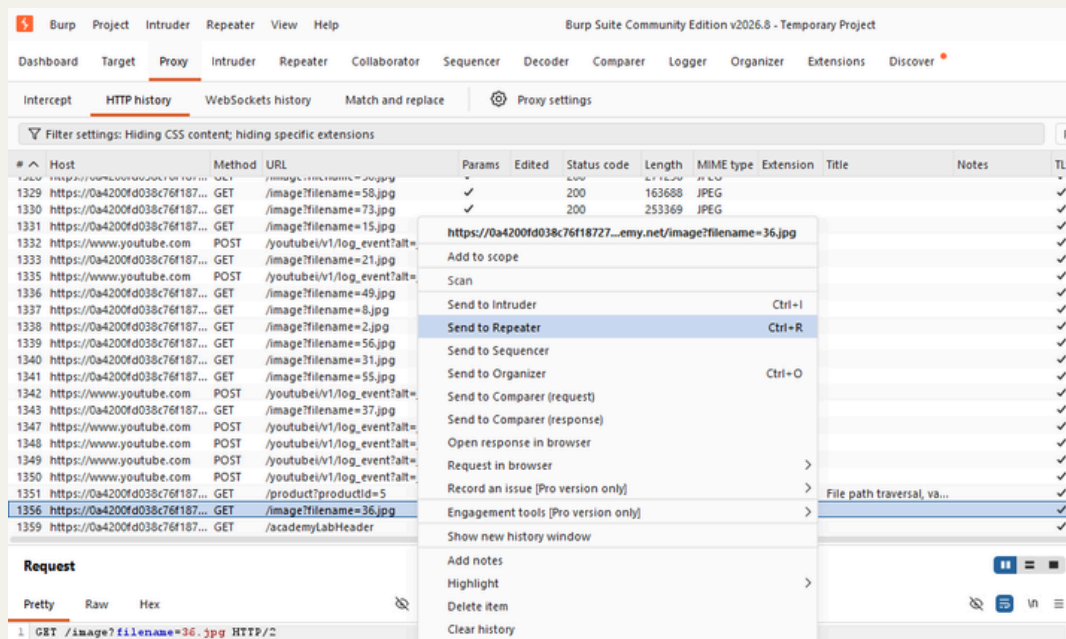


Figura 12.2 Se envió la petición a Repeater para su manipulación

Paso 3: Se seleccionó la solicitud HTTP (GET /image?filename=36.jpg HTTP/2) y se envió al módulo Repeater utilizando el atajo Ctrl + R o el menú contextual para realizar pruebas controladas de manipulación del parámetro.

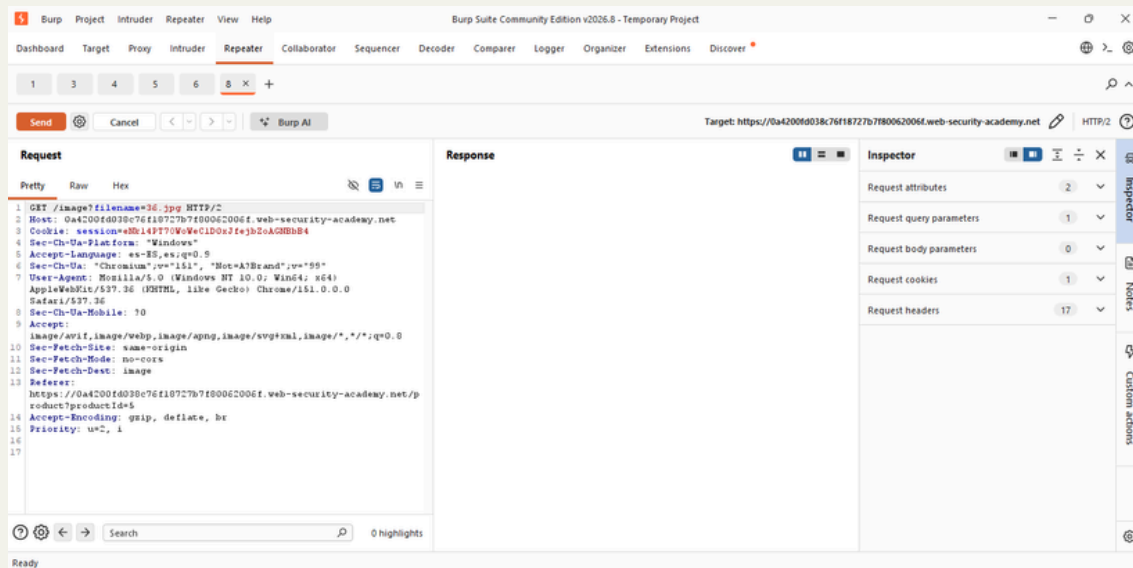


Figura 12.3 Parámetro filename original

Paso 4: Se construyó la carga útil (payload) modificando el valor de filename. Para evadir la regla que exige una extensión válida al final, se inyectaron secuencias de retroceso seguidas de la ruta del archivo objetivo, un byte nulo codificado en URL (%00) y la extensión requerida: ../../../../etc/passwd%00.png.

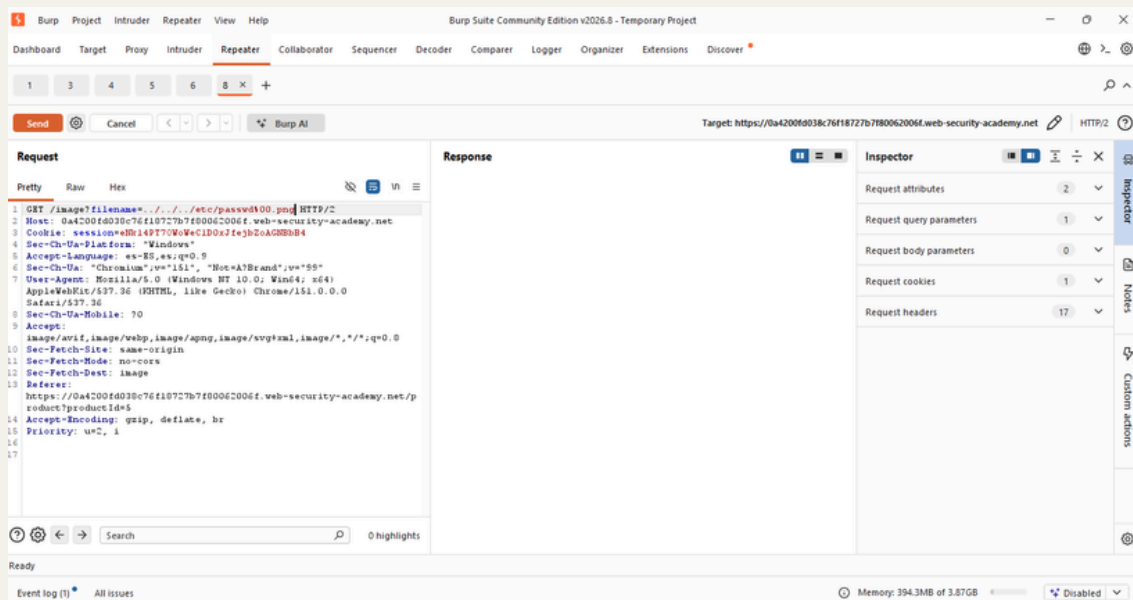


Figura 12.4 Parámetro filename modificado

Paso 5: Se emitió la solicitud manipulada y se analizó la respuesta HTTP devuelta por el servidor (código 200 OK), constatando la extracción exitosa del contenido del archivo /etc/passwd en el cuerpo de la respuesta, lo cual confirma la vulnerabilidad.

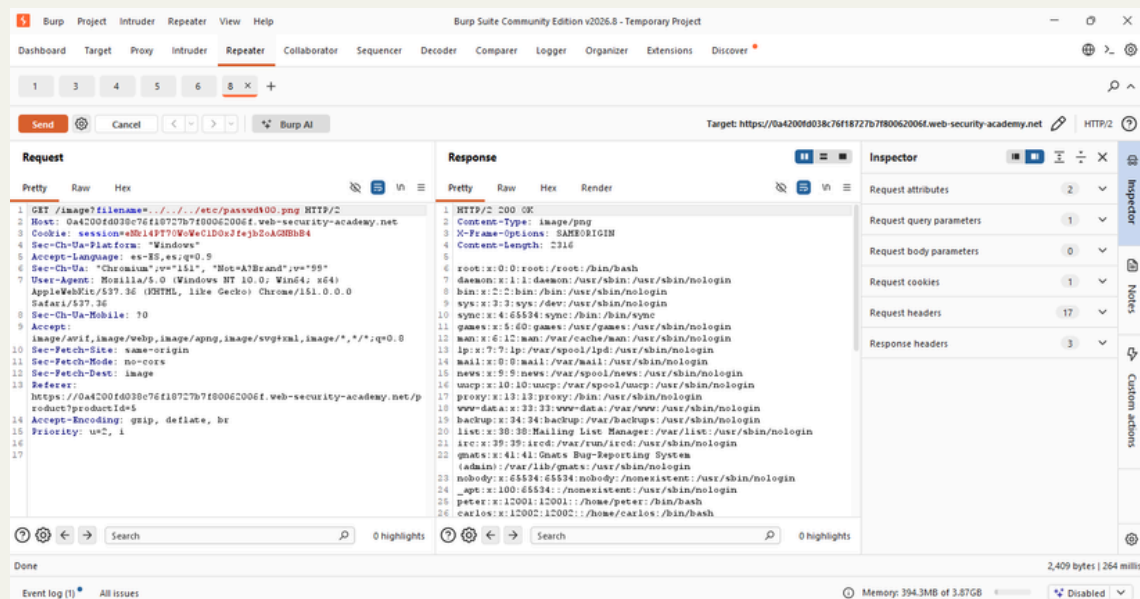


Figura 12.5 Respuesta del servidores al filname modificado

Paso 5: Se analizó la respuesta HTTP, la cual devolvió el contenido de un archivo restringido, validando teóricamente la vulnerabilidad.

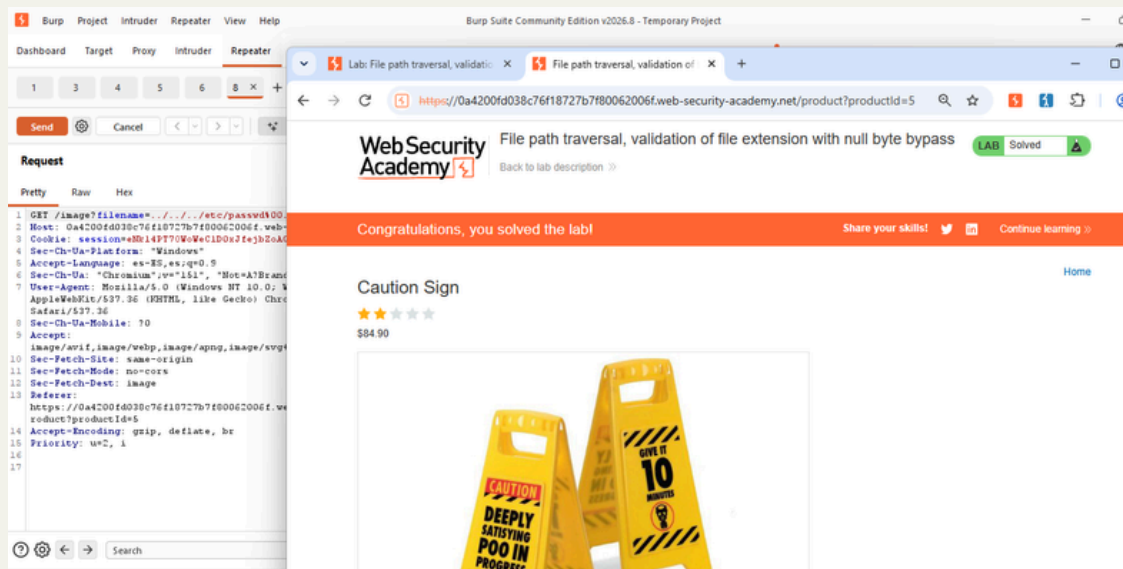


Figura 11.6 Laboratorio finalizado

EXPLICACIÓN DEL PAYLOAD

Payload utilizado:

`../../../../etc/passwd%00.png`

1. Fase de validación de la aplicación: La aplicación web evalúa la cadena completa. Dado que los últimos 4 caracteres coinciden exactamente con `.png`, la comprobación resulta verdadera y la petición es autorizada.
2. Fase de ejecución del sistema operativo: La cadena se entrega a las funciones del núcleo o librerías del sistema de archivos en C. El compilador/librería interpreta `%00` (byte `0x00`) como el carácter terminador de cadena (null terminator). Como consecuencia, la lectura del nombre del archivo se detiene antes del byte nulo, intentando abrir literalmente `../../../../etc/passwd`.
3. Resolución de ruta: El sistema operativo resuelve los saltos relativos (`../../../../`) retrocediendo hasta el directorio raíz (`/`) y devuelve el contenido del archivo de cuentas del sistema, saltándose la restricción de tipo de archivo impuesta en la capa de software.

MITIGACIÓN RECOMENDADA

Para corregir y prevenir ataques de Path Traversal con inyección de caracteres nulos, se deben seguir las siguientes medidas de defensa en profundidad (OWASP Foundation, 2023; CISA, 2022):

1. Actualización de versiones y saneamiento de entradas: Asegurarse de utilizar versiones modernas de los lenguajes de desarrollo (por ejemplo, PHP $\geq$ 5.3.4 solucionó la vulnerabilidad de inyección de byte nulo a nivel de API de sistema de archivos). Asimismo, rechazar explícitamente o limpiar cualquier entrada que contenga bytes nulos (`%00`, `\0`, `0x00`).
2. Canonicalización estricta de rutas: Resolver la ruta canónica y absoluta del archivo mediante funciones seguras del entorno (ej. `realpath()` en PHP, `Path.GetFullPath()` en C#, `os.path.realpath()` en Python) antes de realizar cualquier validación de sufijo o acceso.
3. Verificación de contención de directorio: Validar que la ruta canónica obtenida comience indefectiblemente con el prefijo del directorio base autorizado.
4. Identificadores indirectos (Whitelist / Mapeo seguro): Eliminar por completo el paso de rutas y nombres de archivos controlados por el usuario. Se recomienda mapear las imágenes a identificadores numéricos o UUIDs almacenados en una base de datos o utilizar una lista blanca fija de archivos accesibles.

REFERENCIAS

- CISA. (2022). Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'). Cybersecurity and Infrastructure Security Agency. <https://www.cisa.gov/uscert/bsi/articles/knowledge/coding-practices/path-traversal>
- MITRE. (2023). CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'). Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>
- OWASP Foundation. (2023). Path Traversal. Open Worldwide Application Security Project. https://owasp.org/www-community/attacks/Path_Traversal